Vishnu Waman Thakur Charitable Trust's

VIVA Institute of Technology

Approved by AICTE, New Delhi, Recognized by DTE, Govt. of Maharashtra,
Affiliated to University of Mumbai

AN AUTONOMOUS INSTITUTE

SEMESTER –

Academic Year : _____

Name : _____ Roll No : _____

Subject : _____

Branch : _____ Division : _____Batch : _____

Examination Seat No :_____

Vishnu Waman Thakur Charitable Trust's

VIVA Institute of Technology

Approved by AICTE, New Delhi, Recognized by DTE, Govt. of Maharashtra,
Affiliated to University of Mumbai

AN AUTONOMOUS INSTITUTE

Academic Year 2026-27

CERTIFICATE

This is to certify that Mr./Ms._____

Roll No. _____ Exam Seat No. _____Branch _____

Semester _____Academic Year _____has completed Term work (Practicals / Tutorials / Assignments /Test)in Subject_____ as per the syllabus prescribed by the University of Mumbai successfully.

Faculty

Mini Project on

“Legal Contract Clause Extractor”

Submitted in partial fulfillment of the requirements of the degree

of

Bachelor of Engineering in Computer Science & Engineering (AI & ML) Department

Submitted by

Atharv Bowlekar (02)

Under the guidance of:

Ms. Apurva Jadhav



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(AI & ML) VIVA INSTITUTE OF TECHNOLOGY

At Shirgoan, Virar (East), Tal. Vasai, Dist. Palghar – 401305

2026-27

Table of Contents

Sr No.	Topics	Page No.
1)	Introduction	
2)	Motivation	
3)	Objective	
4)	Resources & Technology Stack Used	
5)	Algorithm & Processing Logic	
6)	Flowchart	
7)	Output & Interface Briefing	
8)	Conclusion & Future Scope	

INTRODUCTION

“Concise briefing of the project topic”

The Legal Contract Intelligence (LCCE) platform is an advanced, full-stack web application designed to apply state-of-the-art Natural Language Processing (NLP) to the legal domain. Historically, the legal field has been highly resistant to automation due to the dense, verbose, and structurally complex nature of legal jargon (often referred to as "legalese"). However, with the advent of Generative AI and Large Language Models (LLMs), it is now possible to programmatically parse, understand, and extract deep semantic meaning from these texts.

LCCE acts as an automated "AI Legal Analyst." The system is engineered to ingest raw unstructured data either via direct text input or by parsing binary PDF documents and process it through a robust backend API. Utilizing a high-performance Generative AI model (Google Gemma), the system systematically deconstructs the contract. It extracts critical metadata (parties involved, monetary values, dates), identifies and categorizes binding obligations, and most importantly, flags predatory or asymmetrical risks. This parsed data is dynamically rendered onto a highly interactive, responsive frontend dashboard utilizing canvas-based data visualizations, and can ultimately be compiled into a professional, downloadable PDF intelligence report.

MOTIVATION

The core motivation for this project stems from a profound societal issue: the inherent and systemic power imbalance in modern legal agreements.

Every day, average consumers, freelancers, and small business owners are forced to sign complex documents such as Master Services Agreements, NDAs, Commercial Leases, and Software EULAs. Large corporations employ dedicated legal teams to draft these contracts in a way that heavily insulates them from liability while placing maximum burden on the signee. Because the average individual lacks the financial resources to hire legal counsel (which can cost upwards of $300 to $500 an hour) for every document they sign, they routinely agree to predatory terms blindly.

These predatory terms often include :

- **Asymmetrical Liability Caps :** Where the corporation is protected from all damages, but the consumer is fully liable.
- **Unilateral Fee-Shifting :** Clauses dictating that if a dispute occurs, the consumer must pay the corporation's legal fees regardless of who is at fault.
- **Auto-Renewal Traps & Predatory Audits :** Hidden clauses that trigger massive financial penalties without fair warning.

The motivation behind LCCE is to democratize legal comprehension. By providing a free, instant, and highly accurate NLP tool, we can translate dense legal jargon into a "Plain English" summary and explicitly warn users of hidden dangers, effectively giving them the power of an expert legal review in seconds.

OBJECTIVE

To successfully build the LCCE platform, the project was broken down into several granular technical objectives:

i. **Unstructured Data Ingestion:** Develop a secure frontend mechanism to handle file uploads, specifically parsing binary PDF files and extracting raw text strings without losing structural integrity.

ii. **Prompt Engineering & JSON Schema Enforcement:** Generative AI models naturally output conversational text. A major objective was to engineer a strict system prompt that forces the LLM to return data perfectly formatted to a predefined JSON schema, ensuring the backend Python code can parse it programmatically without crashing.

iii. **NLP Pipeline Integration:** Successfully integrate a Generative LLM via the OpenRouter API to perform Named Entity Recognition (NER), text classification, and sentiment/risk analysis.

iv. **Data Visualization UI/UX:** Design an enterprise-grade, dark-mode (OLED pitch-black) frontend. Utilize Chart.js to render complex data into intuitive visualizations, specifically a Doughnut chart for the Overall Risk Score and a multi-axis Radar chart for contract Favorability.

v. **Dynamic PDF Compilation:** Implement a backend routing system capable of taking the parsed JSON intelligence data, injecting it into an HTML/CSS template, and converting it into a styled, downloadable A4 PDF report using CSS2.1 paradigms.

RESOURCES AND TECHNOLOGY STACK USED

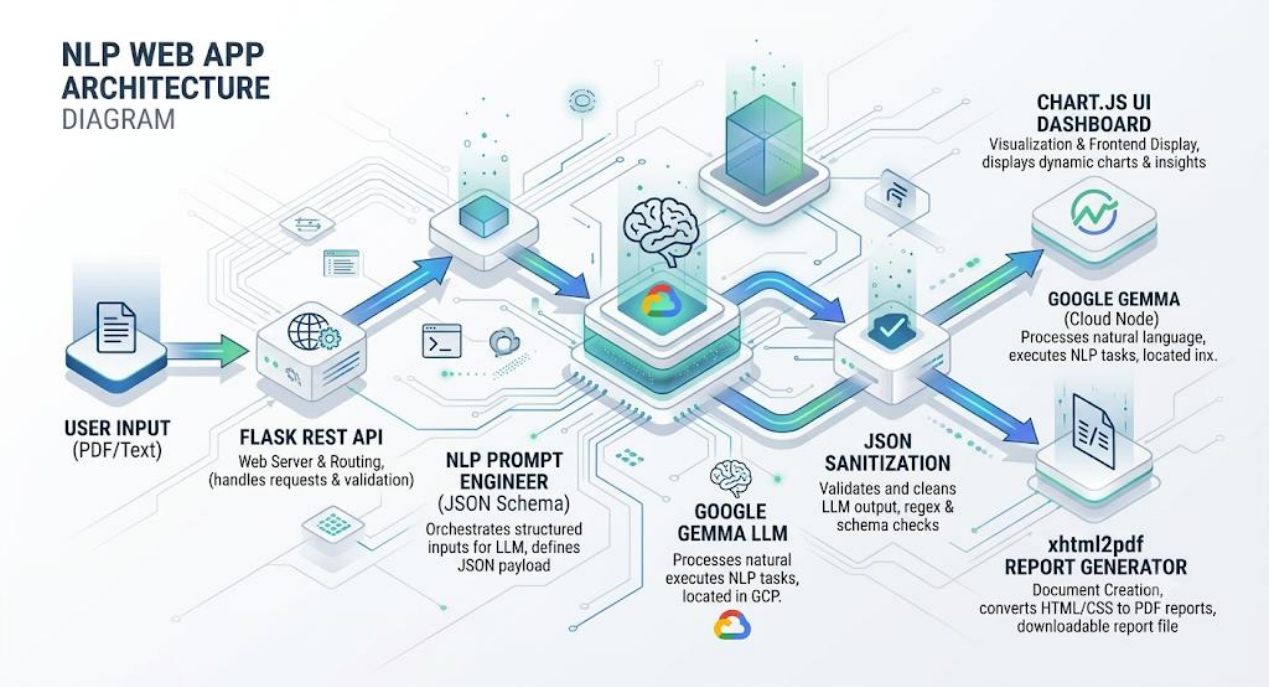
The project was built utilizing a modern, decoupled architecture :

- **Frontend Technologies:**
  - **HTML5 / CSS3:** Utilized advanced custom CSS properties (variables), flexbox/grid layouts, and cubic-bezier transition animations for a premium user experience.
  - **Vanilla JavaScript (ES6+):** Used for DOM manipulation, asynchronous fetch() API calls, and drag-and-drop file handling.
  - **Chart.js:** A robust canvas-based charting library used to render the Intelligence Dashboard graphs.
- **Backend Technologies:**
  - **Python (v3.11):** The core programming language used for server-side logic.
  - **Flask:** A lightweight WSGI web application framework used to build the RESTful API endpoints (/api/analyze and /api/export-pdf).
  - **xhtml2pdf:** A Python library utilized for dynamically converting HTML templates into formatted PDF files.
- **NLP & Artificial Intelligence:**
  - **OpenAI Python SDK:** Utilized as the structural client to communicate with the AI model.
  - **OpenRouter API:** Acts as the routing gateway to access massive open-weight models.
  - **Google Gemma (gemma-4-26b-a4b-it):** The specific LLM chosen for its blazing-fast inference speed, deep reasoning capabilities, and adherence to strict JSON formatting instructions.

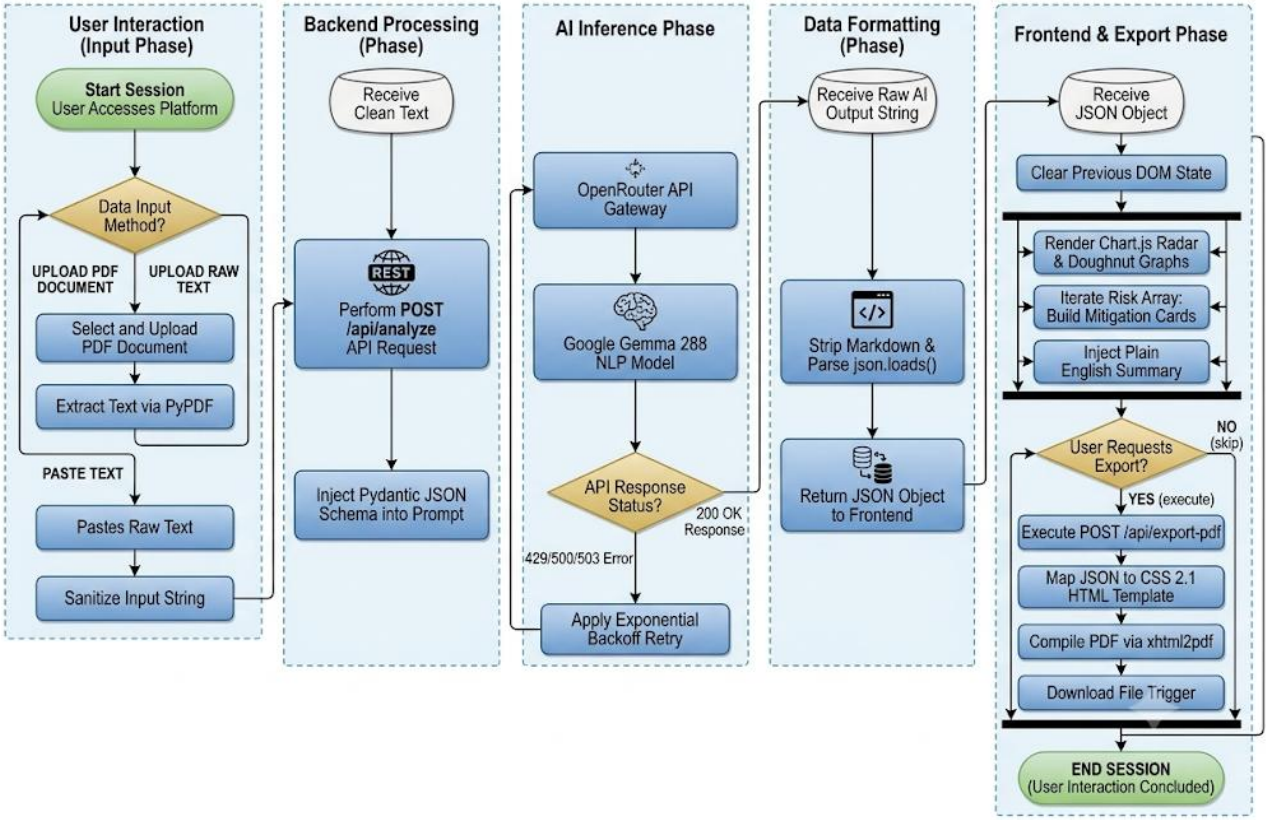
ALGORITHM & PROCESSING LOGIC

Step-by-Step Algorithmic Flow:

1) **Initialization:** The user interface loads. The system waits for an onDrop or onChange event from the file input, or a text paste event.
2) **Payload Construction:** Upon clicking "Analyze", JavaScript intercepts the data. If a PDF is uploaded, it is appended to a FormData object; otherwise, the raw text is used.
3) **Backend Reception:** The Flask route /api/analyze receives the POST request.
4) **Schema Injection:** The backend generates a strict Pydantic BaseModel schema representing the exact data structure required (Risks, Metadata, Favorability, Clauses). This schema is injected into the system prompt.
5) **Inference Execution:** The payload is sent to the Google Gemma model via OpenRouter.
6) **Retry Logic (Resiliency):** If the API returns a 429 (Rate Limit) or 503 (Service Unavailable) error, an exponential backoff algorithm triggers, pausing the thread and retrying the connection up to a maximum limit.
7) **Data Sanitization:** The AI returns a string. The backend strips any markdown formatting (e.g., json) and uses json.loads()` to convert it into a Python dictionary.
8) **Frontend Rendering:** The JSON is returned to the client with a 200 OK status. The JS destroys any old Chart.js instances, creates new ones, iterates through the risk arrays to build DOM elements, and applies highlighting algorithms to the original text.



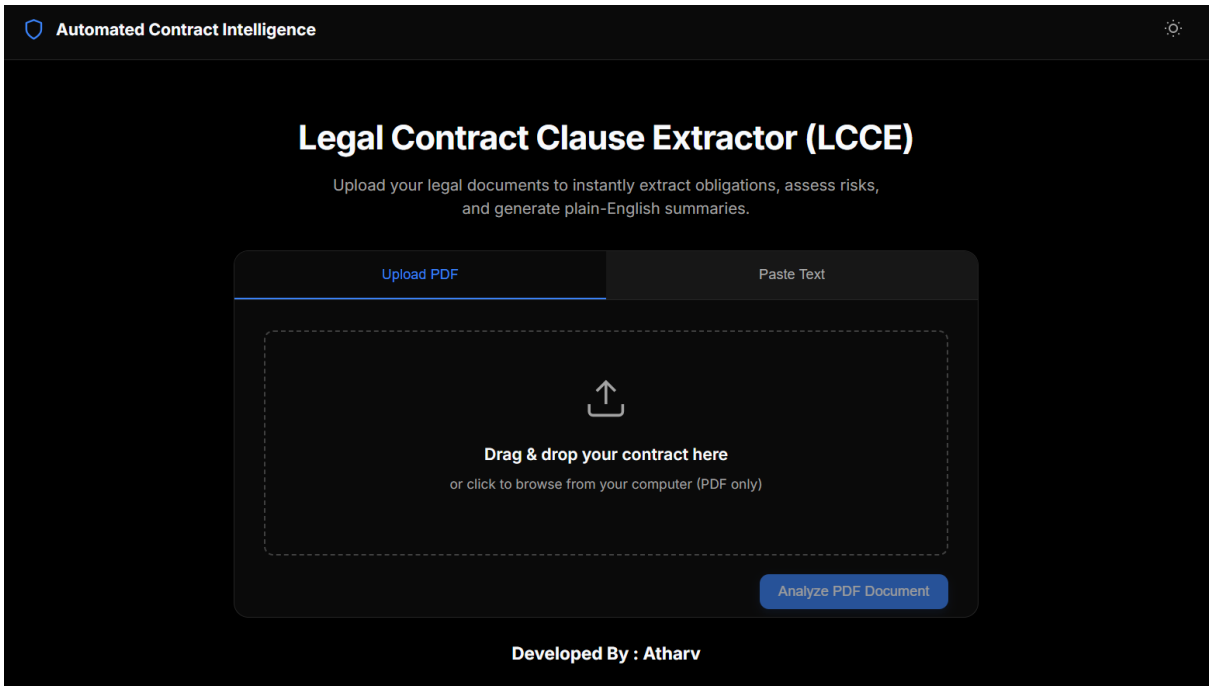
FLOWCHART

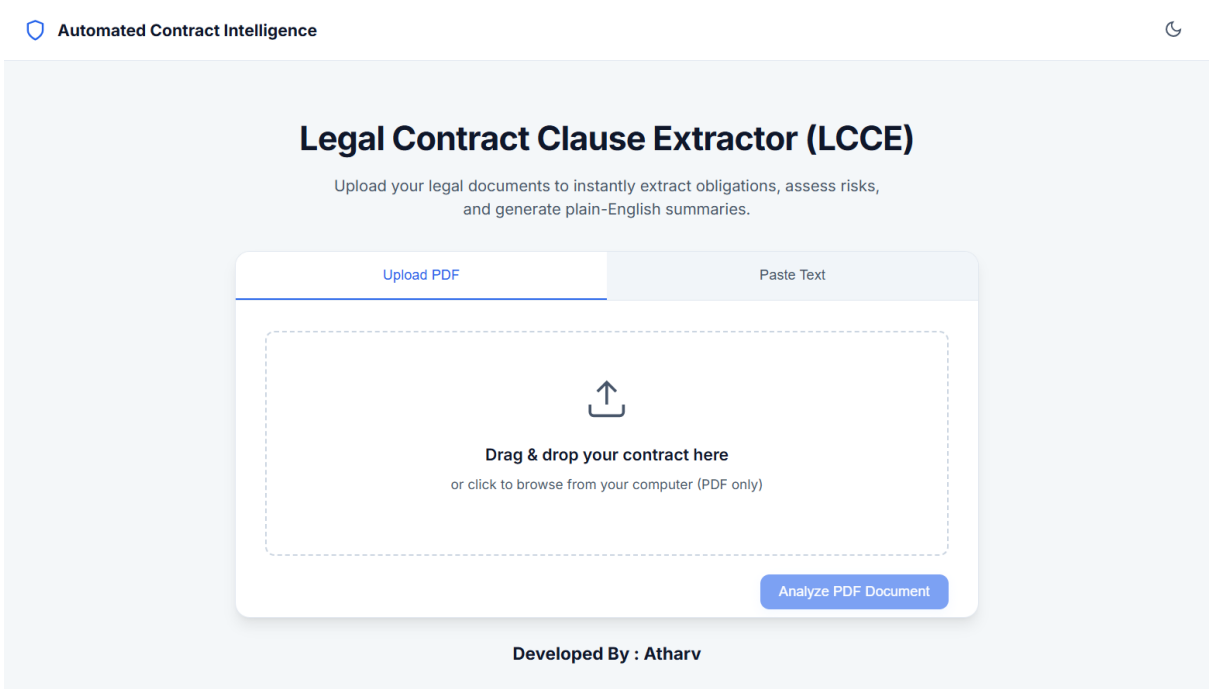

User Interaction (Input Phase)
Start Session
User Accesses Platform
Data Input Method?
UPLOAD PDF DOCUMENT
UPLOAD RAW TEXT
Select and Upload PDF Document
Extract Text via PyPDF
PASTE TEXT
Pastes Raw Text
Sanitize Input String
Backend Processing (Phase)
Receive Clean Text
REST
Perform POST /api/analyze API Request
Inject Pydantic JSON Schema into Prompt
AI Inference Phase
OpenRouter API Gateway
Google Gemma 288 NLP Model
API Response Status?
200 OK Response
429/500/503 Error
Apply Exponential Backoff Retry
Data Formatting (Phase)
Receive Raw AI Output String
Strip Markdown & Parse json.loads()
Return JSON Object to Frontend
Frontend & Export Phase
Receive JSON Object
Clear Previous DOM State
Render Chart.js Radar & Doughnut Graphs
Iterate Risk Array: Build Mitigation Cards
Inject Plain English Summary
User Requests Export?
NO (skip)
YES (execute)
Execute POST /api/export-pdf
Map JSON to CSS 2.1 HTML Template
Compile PDF via xhtml2pdf
Download File Trigger
END SESSION
(User Interaction Concluded)

OUTPUT & INTERFACE BRIEFING

The finished LCCE application provides a highly polished, interactive Graphical User Interface (GUI).

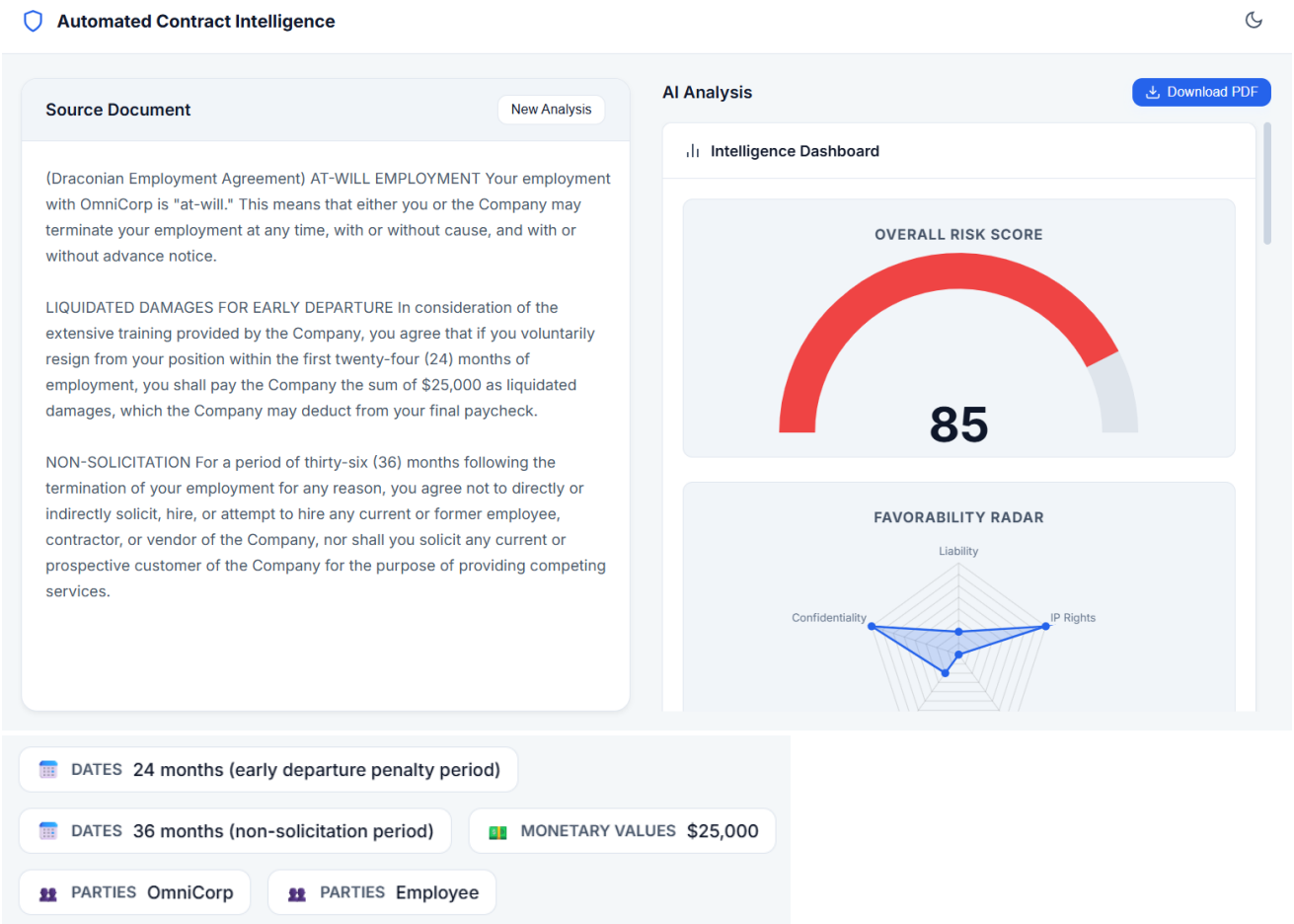
✓ **The Input View:** Features a drag-and-drop zone with animated borders and a clean text-area, styled in a deep OLED pitch-black dark theme.



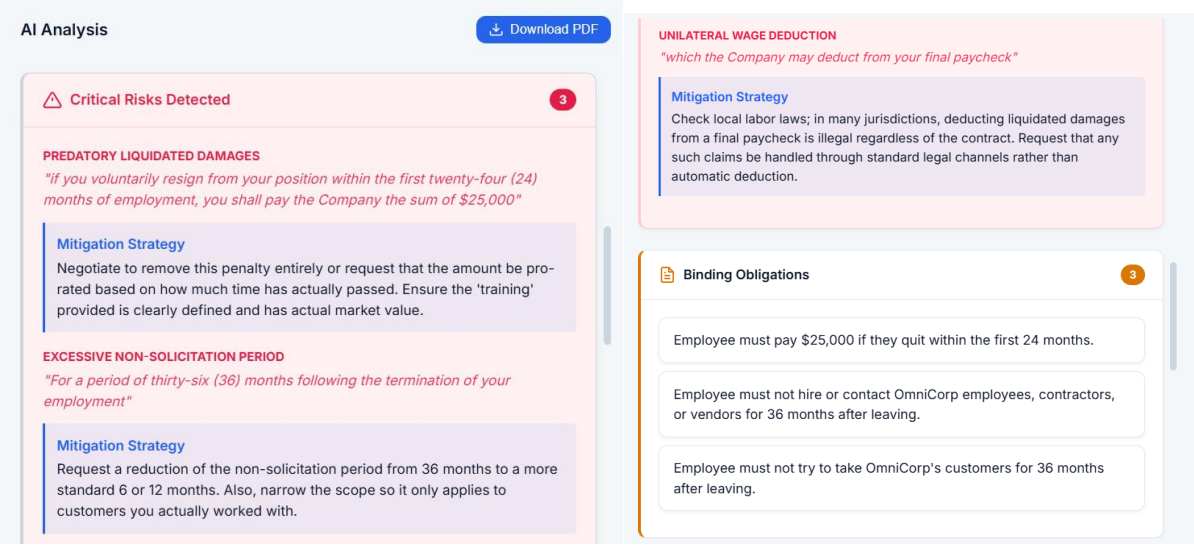


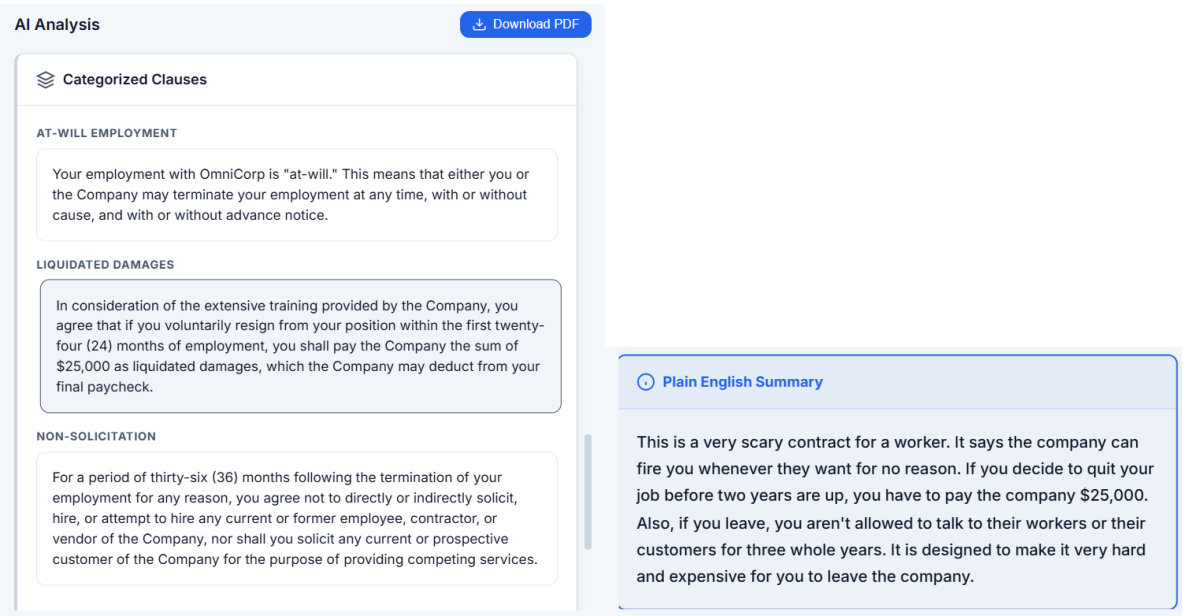
✓ **The Intelligence Dashboard:** Upon successful analysis, the view splits into two main panes:

- **Visual Analytics Pane:** Displays a massive color-coded (Red/Yellow/Green) numerical Risk Score. Below it sits a dynamic Radar Chart that plots the contract's favorability across five distinct legal axes (Liability, IP Rights, Financials, Termination, Confidentiality).



- **Insights Pane:**
  - **Metadata Chips:** Highlights extracted entities (Dates, Monetary Values, Parties) using pill-shaped UI chips.
  - **Executive Summary:** A highlighted, glowing card containing a "10-year-old level" plain-English breakdown of the contract's true meaning.
  - **Risk Cards:** A list of critical risks extracted from the text. Each risk includes the exact offending quote, categorized by risk type, and features a blue "Mitigation Strategy" box advising the user on how to negotiate or fix the clause.





✓ **PDF Reporting Engine:** Clicking the download button triggers a backend compilation process that takes all visual insights and arranges them into a professional, multi-page, heavily formatted A4 PDF report, suitable for printing or emailing to a legal team.

CONCLUSION AND FUTURE SCOPE

The LCCE Mini-Project was executed with overwhelming success, meeting and exceeding all defined technical and functional objectives. By bridging a modern Flask/Vanilla JS stack with cutting-edge Generative NLP (Google Gemma), the system successfully automated a task that traditionally requires hours of expensive human labor.

Key Achievements:

- Successfully forced unstructured LLM outputs into highly structured, programmatically parsable JSON schemas.
- Created a resilient backend capable of handling API failures gracefully without crashing the user session.
- Designed an enterprise-grade UI that visualizes complex legal vectors in an easily digestible format.

Future Scope: Moving forward, the project could be expanded by implementing *Retrieval-Augmented Generation (RAG)*. By integrating a vector database (such as Pinecone or Milvus), the AI could compare the uploaded contract against thousands of standard industry contracts to identify "missing clauses" (e.g., pointing out that an NDA is missing a standard time-limit clause). Additionally, fine-tuning a smaller, dedicated local model (like Llama-3-8B) exclusively on legal data could remove the dependency on external cloud APIs entirely, ensuring maximum data privacy for sensitive documents.